Proiect C - Aprovizionare și stocuri

Ce este comun celor trei proiecte (rulări, chei de idempotență, reîncercări, reporniri, livrabile, notare) este în „01 Ghid student”.

Situația

O firmă românească de producție are un depozit cu câteva sute de articole: mănuși, etichete, rulmenți, ulei hidraulic. Fiecare articol are un cod `sku` și este plătit din bugetul unui centru de cost (departament).

În fiecare dimineață, un cumpărător decide manual ce articole se reaprovizionează, de la ce furnizor, în ce cantitate și la ce preț. Greșelile se repetă:

- un articol aflat deja pe drum este comandat din nou;
- se alege un furnizor blocat sau o ofertă expirată;
- cantitatea nu respectă ambalajul sau comanda minimă;
- o scumpire mare trece neobservată;
- bugetul se termină pe articole necritice, iar depozitul află de lipsa unui articol critic abia când raftul e gol.

Ce aveți de făcut

- La fiecare pornire, automatizarea voastră procesează toate articolele din `GET /items`.
- Fiecare articol primește exact o decizie: `ORDER_CREATED`, `NO_ACTION` sau `MANUAL_REVIEW`, cu codurile de motiv și, dacă s-a ales o ofertă, furnizorul, cantitatea, prețul și costul.
- Fiecare furnizor cu articole de livrat primește exact o comandă de achiziție nouă, cu toate articolele lui.
- Furnizorul, managerul de achiziții și șeful depozitului primesc notificările lor, fiecare o singură dată.
- La final trimiteți raportul cu rezultatul fiecărui articol.
- Parametrii vin din `GET /config`, iar „azi” este `business_date` din `GET /context`, niciodată ceasul calculatorului.
- Gata înseamnă: toate cele de mai sus, cu sume exacte, și același rezultat final dacă automatizarea este oprită oricând și pornită din nou pe același `BASE_URL`.

Fluxul de lucru

1. Citiți `GET /context` (`run_id`, `business_date`) și `GET /config`.
2. Citiți `GET /items`, `GET /stock`, `GET /demand`, `GET /suppliers`, `GET /cost-centers`, `GET /fx-rates` și `GET /purchase-orders`.
3. Citiți ofertele fiecărui articol care nu este retras: `GET /offers?sku=` (le puteți sări pe cele ale articolelor peste punctul de reaprovizionare).
4. Stabiliți în memorie decizia fiecărui articol, în ordinea canonică, cu C-R01 până la C-R09. Nu scrieți nimic încă.
5. Creați câte o comandă pentru fiecare furnizor cu articole `ORDER_CREATED`: `POST /purchase-orders` (C-R10, C-R12).
6. Înregistrați câte o decizie pentru fiecare articol: `POST /replenishment-decisions` (C-R13).

7. Trimiteți notificările: `POST /notifications` (C-R14).
8. Trimiteți raportul: `POST /report`. Ieșiți cu codul 0.

Comenzile se creează înaintea deciziilor pentru că decizia `ORDER_CREATED` conține `po_number`. În rest, ordinea scrierilor nu se notează. În UiPath, țineți `spent` într-un `Dictionary<String, Decimal>` pe centru de cost și liniile comenzilor într-un dicționar pe furnizor.

Regulile

Notații: `ceil(x)` este cel mai mic întreg mai mare sau egal cu x (`ceil(63.5)` = 64, `ceil(64)` = 64). `round2(x)` rotunjește la 2 zecimale, half-up (14.925 → 14.93, 0.625 → 0.63). „Data” înseamnă `business_date`.

ID	Verificare	Rezultat	Cod
C-R01	`discontinued` este `true`. Oprire.	`NO_ACTION`	`ITEM_DISCONTINUED`
C-R02	Poziția de stoc = `on_hand - reserved + incoming`. `incoming` = suma `quantity - received_quantity` pe liniile articolului din comenzile `OPEN` cu `eta <= data + horizon_days` (zile calendaristice), inclusiv cele cu `eta` în trecut. Comenzile `CLOSED` și `CANCELLED` nu contează.	pas de calcul	
C-R03	Punct de reaprovizionare = `ceil(avg_daily_demand × reference_lead_time_days) + safety_stock`. Poziție mai mare decât punctul: oprire. Altfel articolul se comandă.	`NO_ACTION`	`ABOVE_REORDER_POINT`
C-R04	`need = max_stock - poziție`; `base = ceil(need / pack_size) × pack_size`. Pentru fiecare ofertă: `quantity = max(base, ceil(moq / pack_size) × pack_size)`.	pas de calcul	
C-R05	Oferta este eligibilă dacă: furnizorul are `status ACTIVE`; `valid_from <= data <= valid_to` (`valid_to null` = fără sfârșit); `lead_time_days <= max_lead_time_days[criticality]`; `quantity <= max_qty`. Nicio ofertă eligibilă sau nicio ofertă: oprire.	`MANUAL_REVIEW`	`NO_ELIGIBLE_SUPPLIER`
C-R06	Pentru fiecare ofertă eligibilă: `unit_price` = prețul treptei cu cel mai mare `min_qty <= quantity`; `rate` = 1 pentru RON, iar pentru EUR cursul din data de lucru sau, dacă lipsește, cel din cea mai recentă zi anterioară; `total_cost_ron = round2(quantity × unit_price × rate)`.	pas de calcul	
C-R07	Se alege oferta cu cel mai mic `total_cost_ron`, apoi cel mai mic `lead_time_days`, apoi cel mai mare `on_time_rate` al furnizorului, apoi cel mai mic `supplier_id` (comparat ca text).	pas de calcul	
C-R08	`last_purchase_price_ron` nu este `null` și `unit_price × rate > last_purchase_price_ron × (1 + price_increase_review_pct / 100)`, calculat exact, fără rotunjire.	`MANUAL_REVIEW`	`PRICE_INCREASE_REVIEW`
C-R09	`spent + total_cost_ron > budget_ron` pentru centrul de cost al articolului.	`MANUAL_REVIEW`	`BUDGET_EXCEEDED`

ID	Verificare	Rezultat	Cod
C-R10	Nicio regulă de mai sus nu se aplică. După ce toate articolele sunt decise, fiecare furnizor cu articole `ORDER_CREATED` primește o comandă cu exact acele articole, liniile sortate după `sku`.	`ORDER_CREATED`	niciunul
C-R11	Comenzile cu alt `run_id` sau cu `run_id` `null` contează în C-R02. Comenzile cu `run_id`-ul din `GET /context` (create de voi înainte de o repornire) nu contează. Nicio comandă nu se scade din `budget_ron`: cele anterioare sunt deja scăzute.	pas de calcul	
C-R12	`POST /purchase-orders` pentru un furnizor primește 500 la toate cele 3 încercări. Se aplică tuturor articolelor acelei comenzi.	`MANUAL_REVIEW`	`PO_CREATION_FAILED` (singurul cod)
C-R13	Exact o decizie pentru fiecare articol, înregistrată după C-R10 și C-R12.	scriere	
C-R14	Notificările din secțiunea „Notificări”.	scriere	

Precedența și limitele

- Limitele sunt incluse: un `eta` exact în ziua `data + horizon_days` contează (o zi mai târziu nu), iar poziția egală cu punctul înseamnă comandă. Sunt acceptate termenul egal cu limita (7 zile la `A`), `quantity` egală cu `max_qty`, prețul egal cu limita C-R08 (ultimul preț 10.00, 15%: 11.50 trece, 11.51 nu) și bugetul atins exact. Cantitatea egală cu `min_qty` primește prețul acelei trepte.
- C-R01, C-R03 și C-R05 opresc evaluarea articolului. După alegerea ofertei se evaluează și C-R08, și C-R09; dacă ambele eșuează, decizia are ambele coduri. Dacă oferta aleasă eșuează, nu treceți la alta.
- Ordinea canonică: criticitate `A`, `B`, `C`, apoi `sku` crescător (comparat ca text). `GET /items` este sortat doar după `sku`.
- `spent` adună `total_cost_ron` al articolelor `ORDER_CREATED` ale unui centru de cost și pornește de la 0 la fiecare pornire. Un articol `MANUAL_REVIEW` nu consumă buget, deci un articol următor mai ieftin poate încăpea. C-R12 nu restituie bugetul și nu schimbă alte articole.
- Cantitatea poate depăși `need` din cauza ambalajului și a MOQ. Nu se reduce la `max_qty`, iar un articol nu se împarte între furnizori.

Exemplu

Data de lucru 2026-11-16, configurația publică, curs EUR 4.9750. Articolul are criticitate `B`, `pack_size` 10, `safety_stock` 40, `max_stock` 300, `reference_lead_time_days` 10, ultimul preț 19.80 RON, cerere 6.35 pe zi, 75 în stoc, 20 rezervate. Comenzi `OPEN`: o linie de 60 (25 primite) cu `eta` 2026-11-22 și una de 100 cu `eta` 2026-12-24. Ofertele vin de la furnizorii S1, S2 și S3.

1. Orizontul se termină pe 2026-12-16, deci contează doar prima linie: poziția = 75 - 20 + 35 = 90.
2. Punctul = ceil(6.35 × 10) + 40 = 64 + 40 = 104. Cum 90 <= 104, se comandă: `need` = 210, `base` = 210.
3. S1 (RON, MOQ 1, 25.40, termen 4 zile): 210 bucăți, 5334.00 RON.
4. S2 (EUR, MOQ 250, 4.2500 de la 250 de bucăți, termen 9 zile): 250 bucăți, 250 × 4.2500 × 4.9750 = 5285.9375, deci 5285.94 RON.
5. S3 (RON, 20.90, termen 16 zile) depășește limita de 14 zile pentru `B`: neeligibil, deși e cel mai ieftin.
6. Se alege S2. Prețul 4.2500 × 4.9750 = 21.14375 nu depășește 19.80 × 1.15 = 22.77.

7. Centrul de cost are 20000.00 RON, din care articolele `A` au consumat 12400.00. Cum 12400.00 + 5285.94 = 17685.94 <= 20000.00, rezultatul este `ORDER_CREATED`.

Decizia: furnizorul S2, `quantity` 250, `unit_price` `"4.2500"`, `currency` `EUR`, `total_cost_ron` `"5285.94"`, `po_number` al comenzii lui S2 din această rulare.

Notificări

Eveniment	Destinatar (`recipient_role`, `recipient`)	`template`	Câmp în corp
S-a creat o comandă în această rulare	`SUPPLIER`, `email` al furnizorului	`po_created`	`po_number`
Articol cu rezultat final `MANUAL_REVIEW`, orice motiv	`PROCUREMENT_MANAGER`, `notification_recipients.PROCUREMENT_MANAGER`	`replenishment_review`	`sku`
Articol cu rezultat final `MANUAL_REVIEW` și criticitate `A`	`WAREHOUSE_MANAGER`, `notification_recipients.WAREHOUSE_MANAGER`	`stockout_risk`	`sku`

- Fiecare notificare se trimite exact o dată, după ce rezultatul final al articolului este cunoscut, adică după crearea comenzilor (C-R12 poate schimba rezultatul).
- `po_created` se trimite o dată pentru fiecare comandă, nu pentru fiecare linie.
- Nimic altceva: nicio notificare pentru `NO_ACTION` sau `ORDER_CREATED`, nimic pentru comenzile altor rulări, niciun rezumat. Fiecare notificare în plus costă o verificare.
- Corpul: `template`, `recipient_role`, `recipient` și `po_number` (la `po_created`) sau `sku` (la celelalte două).

Situații speciale

- **Comenzi ale unei rulări anterioare.** Datele pot conține comenzi create azi de o rulare care s-a oprit, cu alt `run_id` și numerotate exact ca ale voastre (`PO-261116-001`). Sunt comenzi obișnuite: contează în poziție, nu primesc `po_created` și nu apar în decizii; furnizorul lor primește totuși comanda acestei rulări. Le recunoașteți doar după `run_id`, niciodată după `order_date` sau `po_number`.
- **Răspuns pierdut.** Dacă `POST /purchase-orders` primește 503 deși comanda s-a creat, reîncercarea cu aceeași cheie și același corp primește aceeași `po_number`. O cheie nouă ar crea o a doua comandă.
- **Comandă imposibil de creat (C-R12).** Doar 500 la toate cele 3 încercări; hub-ul răspunde la fel și după o repornire. Deciziile păstrează câmpurile de comandă cu `po_number` `null`, furnizorul nu primește `po_created`, articolele primesc notificările de revizuire. Dacă printre încercări apare un 503 sau un timeout, nu este C-R12: comanda poate exista deja, deci ieșiți cu cod diferit de 0, iar repornirea termină lucrul. La un program corect, asta se întâmplă doar la testele de repornire (503 `RUN_PAUSED`), unde runner-ul repornește programul.

- **Notificare imposibil de trimis.** Doar `POST /notifications` cu 500 la toate cele 3 încercări: săriți notificarea, scrieți-o în log și continuați. Notarea nu o mai așteaptă.
- **422 la `POST /purchase-orders`** (de exemplu `PACK_SIZE_VIOLATION`, `MOQ_VIOLATION`, `MAX_QTY_EXCEEDED`, `NO_VALID_OFFER`, `SUPPLIER_NOT_ACTIVE`) înseamnă un bug în C-R04 sau C-R05, nu un rezultat de business: ieșiți cu cod diferit de 0.
- **Repornire.** Automatizarea repornită recitește tot, nu păstrează nimic local și decide din nou toate articolele, în ordinea canonică, cu `spent` = 0, inclusiv pe cele deja decise. Comenzile cu `run_id`-ul vostru rămân în afara poziției și a bugetului; dacă le-ați număra, deciziile s-ar schimba și hub-ul ar răspunde 422 `IDEMPOTENCY_KEY_REUSED`. Pentru fiecare scriere, ambele variante dau același rezultat: o retrimiteți cu aceeași cheie și același corp (hub-ul răspunde cu replay, deci o comandă primește aceeași `po_number`) sau citiți ce e deja înregistrat (`GET /purchase-orders?supplier_id=`, `GET /replenishment-decisions`, `GET /notifications`) și o săriți.
- **Ordine fixă.** Liniile comenzii după `sku`, `reason_codes` în ordinea din tabelul regulilor (notarea le compară ca mulțime), intrările raportului în ordinea din `GET /items`. Nu corectați o decizie trimițând alta și nu retrimiteți sub o cheie nouă: ambele se notează ca duplicate.
- **Câmpuri necunoscute** (de exemplu `ean`, `note`, `preferred` pe articole, oferte sau furnizori): ignorați-le, oricare le-ar fi conținutul.

Raportul final

`POST /report` cu cheia `report` și corpul `{"items": [...]}`: o intrare pentru fiecare articol, în ordinea din `GET /items`. O intrare are exact câmpurile `sku`, `outcome`, `reason_codes` (poate fi listă goală) și `po_number` (sau `null`) și trebuie să corespundă deciziei.

Fiecare pornire trimite raportul cu aceeași cheie și, după o repornire, cu același corp; contează ultimul raport acceptat. Proiectul C nu folosește cheile `report-2`, `report-3`: un 422 `IDEMPOTENCY_KEY_REUSED` pe raport arată un bug.

Date și formate

Apeluri

Apel	Ce întoarce sau face
`GET /context`	`run_id`, `business_date`
`GET /config`	parametrii regulilor (tabelul următor)
`GET /items`	articolele, sortate după `sku`: `pack_size`, `safety_stock`, `max_stock`, `reference_lead_time_days`, `criticality`, `discontinued`, `last_purchase_price_ron`, `cost_center`
`GET /stock`	`sku`, `on_hand`, `reserved`
`GET /demand`	`sku`, `avg_daily_demand`
`GET /suppliers`	`supplier_id`, `status` (`ACTIVE`, `INACTIVE`, `BLOCKED`), `on_time_rate`, `email`
`GET /offers?sku=`	toate ofertele articolului: `supplier_id`, `currency` (`RON` sau `EUR`), `moq`, `max_qty`, `lead_time_days`, `valid_from`, `valid_to`, `price_tiers` (`min_qty`, `unit_price`)
`GET /cost-centers`	`cost_center`, `budget_ron`: bugetul acestei rulări, cu comenzile anterioare deja scăzute, neschimbat de comenzi sau de reporniri

Apel	Ce întoarce sau face
`GET /fx-rates`	cursurile EUR → RON (`date`, `currency` `EUR`, `rate_ron`) din zilele bancare (luni-vineri) ale ultimelor 10 zile până la data de lucru
`GET /purchase-orders`	comenzile, cu filtrele `status`, `sku`, `supplier_id`: `po_number`, `supplier_id`, `status` (`OPEN`, `CLOSED`, `CANCELLED`), `run_id`, `lines` (`sku`, `quantity`, `received_quantity`, `eta`)
`POST /purchase-orders`	creează o comandă din `supplier_id` și `lines` (`sku`, `quantity`) și întoarce `po_number`. Cheie: `{supplier_id}:po`
`POST /replenishment-decisions`	înregistrează o decizie. Cheie: `{sku}:decision`. `GET` (filtru `sku`) citește deciziile rulării, cu aceleași câmpuri
`POST /notifications`	trimite o notificare. Chei: `{po_number}:po_created`, `{sku}:replenishment_review`, `{sku}:stockout_risk`. `GET` (filtre `sku`, `po_number`) citește notificările rulării, cu aceleași câmpuri
`POST /report`	raportul final. Cheie: `report`

Configurația (`GET /config`)

Cheie	Valoare publică	Regula
`currency`	`RON`, mereu	C-R06, C-R09
`horizon_days`	30	C-R02
`price_increase_review_pct`	`"15.00"`	C-R08
`max_lead_time_days`	A 7, B 14, C 30	C-R05
`notification_recipients`	adresele pentru `PROCUREMENT_MANAGER` și `WAREHOUSE_MANAGER`	C-R14

Rulările private pot folosi alte valori pentru toate cheile, în afară de `currency`. Sandbox-ul păstrează valorile publice și data 2026-11-16 (luni): o dată de lucru în weekend și alte valori le verificați cu teste proprii, fără hub (de exemplu teste unitare pe calcule).

Decizia

O decizie are `sku`, `outcome`, `reason_codes`, `po_number` și cele cinci câmpuri de comandă ale ofertei alese: `supplier_id`, `quantity`, `unit_price`, `currency`, `total_cost_ron`.

Rezultat	`reason_codes`	Câmpuri de comandă	`po_number`
`ORDER_CREATED`	`[]`	oferta aleasă	comanda acestei rulări care conține linia
`NO_ACTION`	`ITEM_DISCONTINUED` sau `ABOVE_REORDER_POINT`	`null`	`null`
`MANUAL_REVIEW`	`NO_ELIGIBLE_SUPPLIER`	`null`	`null`
`MANUAL_REVIEW`	`PRICE_INCREASE_REVIEW` și/sau `BUDGET_EXCEEDED`, sau `PO_CREATION_FAILED`	oferta aleasă	`null`

`unit_price` este prețul treptei, în moneda ofertei, în orice scriere zecimală a aceleiași valori. `total_cost_ron` are 2 zecimale. Câmpurile `null` pot lipsi din corp.

Formate și variații

- Sumele, prețurile, cursurile, cererea și `on_time_rate` sunt șiruri zecimale: `"4.5"`, `"4.50"` și `"4.5000"` au aceeași valoare, iar `"7"` este un preț valid. Citiți-le ca zecimale exacte, niciodată ca `float` sau `double`.
- Cantitățile, `pack_size`, `moq`, `max_qty`, termenele și stocurile sunt întregi. Datele (`YYYY-MM-DD`) se compară ca date, identificatorii exact.
- Rotunjiți doar `total_cost_ron`, o singură dată, la final. Rotunjirea prealabilă a prețului în RON sau a costului în EUR dă alt rezultat în unele teste.
- Date lipsă: fără înregistrare de stoc, `on_hand` și `reserved` sunt 0; fără înregistrare de cerere, cererea este 0; `last_purchase_price_ron` `null` înseamnă fără verificare de preț.
- Pot apărea: `reserved` mai mare decât `on_hand` (poziție negativă); cerere `"0"` sau cu până la 4 zecimale; prețuri cu 0 până la 4 zecimale; articole retrase cu stoc, oferte și comenzi deschise; articole fără oferte; `moq` și `max_qty` care nu sunt multipli de `pack_size`; una sau mai multe trepte; mai multe oferte ale aceluiași furnizor pentru același articol, cu perioade care nu se suprapun.
- Mai pot apărea: linii primite parțial sau complet; `eta` în trecut, în orizont sau după el; mai multe comenzi și linii pentru același articol; mai multe centre de cost; o comandă cu linii din centre de cost diferite; data de lucru în weekend (fără curs în acea zi).
- Garantat: `need` >= 1 pentru un articol comandat; prima treaptă nu depășește `moq`; există mereu un curs la data de lucru sau înainte; furnizorul oricărei oferte este în `GET /suppliers`, iar centrul de cost al oricărui articol în `GET /cost-centers`; `max_lead_time_days` are mereu `A`, `B` și `C`; fiecare linie de comandă are `eta` și `received_quantity <= quantity`. În articole și oferte, doar `last_purchase_price_ron` și `valid_to` pot fi `null`. Nimic altceva nu variază.

Ce se notează

- Scenariile publice și private. O rulare publică conține cele 12 scenarii publice (44 de articole) și arată fiecare verificare. Scenariile private aplică aceleași reguli pe alte date de intrare, altă dată de lucru și alte valori de configurare, o dată dintr-o bucată și o dată cu reporniri. Se verifică per articol rezultatul, codurile, decizia unică, câmpurile de comandă, `po_number`, notificările și raportul, iar per furnizor comanda unică, liniile (ordine și cantități) și `po_created`; orice comandă sau notificare în plus costă o verificare.
- Cererea de modificare (CR), primită după predare, cu termen scurt și scenarii proprii.
- Discuția individuală: arătați unde e o regulă în soluție, o modificați pe loc și explicați ce face automatizarea la o repornire. Detaliile sunt în „01 Ghid student”.